AI GROUP PROJECT REPORT

Joshitha Reddy (B24DS009)
Prem Charan (B24DS008)
Sanjana Challa (B24ME012)
V. Vishal (B24EC036)
Y. Lukesh (B24DS039)

PROJECT TITLE

A Comparative Study of Breadth-First Search and Uniform Cost Search for Weighted-Terrain Treasure Hunting

Abstract

This project presents an interactive pathfinding simulator that compares Breadth-First Search (BFS) and Uniform Cost Search (UCS) in a grid-based treasure-hunting environment. The environment contains multiple terrain types with different movement costs, impassable obstacles, a starting cell, a destination, and optional treasure locations. BFS explores the grid in increasing number of moves, whereas UCS expands the lowest cumulative-cost state first. The application visualizes the routes, explored states, search frontiers, and performance metrics, helping users understand the difference between shortest-path and least-cost objectives. A React and TypeScript interface provides interactive simulation and inspection, while a Python Streamlit implementation offers a second way to run the experiment.

Artificial Intelligence, Breadth-First Search, Uniform Cost Search, Pathfinding, Search Algorithms, Treasure Hunt, Streamlit

2 Introduction

Pathfinding is a fundamental problem in Artificial Intelligence (AI). An agent must find a valid route from an initial state to a goal state while respecting the constraints of its environment. In simple environments, minimizing the number of moves may be sufficient. In realistic environments, however, different actions or terrain types can have different costs. The route with the fewest moves may therefore not be the cheapest route.

The project, *PathBench: BFS vs UCS Pathfinding Simulator*, demonstrates this distinction through a treasure-hunt game. A map is represented as a two-dimensional grid. The player starts at a designated cell, may need to collect

treasures, and must finally reach the goal. Grass, mud, and water have different movement costs, while obstacles cannot be crossed. The simulator executes BFS and UCS on the same map and compares their routes and search behaviour.

3 Problem Statement

Given a grid environment with a start cell S, a goal cell G, optional treasure cells T_i, and terrain-dependent movement costs, determine a valid route that visits the required treasures and reaches the goal.

The project compares two objectives:

- **BFS:** Minimize the total number of moves (hops).
- **UCS:** Minimize the sum of movement costs along the route.

The comparison considers path cost, number of steps, expanded nodes, execution time, and (when treasure collection is enabled) net score.

4 Objectives

The main objectives are:

1. Implement BFS and UCS for a two-dimensional grid.
2. Model environments with weighted terrain and blocked cells.
3. Support maps with multiple treasures and a final destination.
4. Visualize the route and the order in which cells are explored.
5. Inspect the BFS FIFO queue and the UCS priority queue.
6. Compare algorithmic performance using automatically generated metrics and a textual verdict.
7. Provide both an interactive React interface and a Python Streamlit implementation.

5 Environment and State Representation

The map is a rectangular grid. Each cell stores its row and column, terrain type, and optional role (start, goal, or treasure). Movement is allowed in four directions: up, right, down, and left. Diagonal movement is not used.

The cost of a route is calculated by adding the cost of each entered cell, excluding the starting cell:

$$C(P) = \sum_{i=1}^{k} c(v_i),$$

Table 1: Terrain model used by the simulator

Terrain	Movement cost	Behaviour
Grass	1	Walkable
Mud	3	Walkable, higher cost
Water	5	Walkable, highest cost
Obstacle	∞	Blocked

where v_i is the i-th cell entered after leaving the start and $c(v_i)$ is its terrain cost. The number of steps is the number of transitions between adjacent cells:

$$H(P) = |P| - 1.$$

The application includes predefined maps such as Meadow Crossing, The Swamp Shortcut Trap, Archipelago and Straits, and The Triple Corridor. It also provides a custom map builder in which users can select grid dimensions and paint terrain, start, goal, and treasure cells.

6 Algorithms

6.1 Breadth-First Search

BFS uses a First-In-First-Out (FIFO) queue. It begins by inserting the start cell. At each iteration, it removes the oldest cell from the queue and inserts all valid, unvisited neighbours. A parent map records the predecessor of each visited cell so that the final route can be reconstructed.

BFS expands states in non-decreasing order of depth. Since each legal move is treated as one unit step, BFS finds a path with the minimum number of steps in an unweighted graph. It does not account for the different terrain costs when choosing which cell to expand.

[language=Python,basicstyle=,frame=single] `BFS(start, goal):  queue = FIFO queue containing start visited = start parent = empty map  while queue is not empty:  current = queue.pop`$_f ront()if current == goal : returnreconstruct(parent, goal)$ `for neighbour in valid`$_n eighbours(current) : if neighbournotinvisited : visited.add(neighbour)parent[neighbour] = currentqueue.push_back(neighbour)$ `return failure`

6.2 Uniform Cost Search

UCS uses a min-priority queue ordered by the cumulative path cost $g(n)$. The start state is inserted with cost zero. When a state is removed from the priority queue, it is the currently cheapest frontier state. For every valid neighbour, UCS calculates the new cumulative cost and updates the best-known cost and parent if the new route is cheaper.

Because all traversable terrain costs are positive, UCS finds a minimum-cost route. Unlike BFS, it can deliberately take more steps to avoid expensive terrain.

[language=Python,basicstyle=,frame=single] UCS(start, goal): priority$_q$ueue.push($start, priority = 0$)$cost[start] = 0parent = emptymapclosed = emptyset$
while priority$_q$ueueisnotempty : $current = pop_lowest_cost()$
if current is already closed: continue add current to closed
if current == goal: return reconstruct(parent, goal)
for neighbour in valid$_n$eighbours($current$) : $new_cost = cost[current]+terrain_cost(neighbour)ifnew_cost < cost[neighbour]$: $cost[neighbour] = new_costparent[neighbour] = currentpriority_queue.push(neighbour, new_cost)$
return failure

6.3 Multi-Treasure Route Planning

When treasure collection is enabled, the route must visit every treasure before reaching the goal. The implementation treats the start, treasure cells, and goal as points of interest. It computes a shortest-step segment for BFS or a least-cost segment for UCS between relevant pairs, caches the segment results, and evaluates possible treasure orders.

For up to six treasures, the implementation evaluates all treasure permutations and selects the feasible order with the lowest algorithm-specific metric. For more than six treasures, it uses a greedy nearest-next-treasure strategy to limit computation. The selected segments are then joined to form the complete route. Therefore, the multi-treasure route is exact with respect to the chosen segment metric for up to six treasures; the greedy strategy for larger counts is a practical approximation.

7 System Design and Implementation

The project contains two application interfaces:

- **React/TypeScript interface:** Built with React, Vite, TypeScript, Tailwind CSS, and Lucide icons. The main application manages map state, algorithm settings, simulation playback, and comparison results.
- **Python Streamlit interface:** Implements the algorithms using Python, NumPy, Matplotlib, `collections.deque`, and `heapq`. It provides a browser-based interface for selecting or generating maps and viewing algorithm comparisons.

The main React modules include:

- `pathfinding.ts`: BFS, UCS, multi-goal route planning, path reconstruction, and simulation-frame generation.
- `priorityQueue.ts`: Binary min-heap priority queue used by UCS.
- `presets.ts`: Predefined grid configurations.
- `verdict.ts`: Generates a comparison summary and tactical insights from the computed results.
- `GridMatrix.tsx`: Displays the map, explored cells, and computed paths.

- `ControlDeck.tsx`: Provides map, terrain, treasure, and playback controls.
- `FrontierInspector.tsx`: Displays the BFS queue and UCS priority queue, or the final route segments.
- `MetricsAudit.tsx`: Presents the performance comparison.

7.1 Application Workflow

The overall workflow is summarized in Fig. 1.

[node distance=5mm, block/.style=draw, rounded corners, align=center, minimum width=0.78minimum height=7mm, font=, arrow/.style=-Stealth[length=2mm], thick] [block] (input) Select preset or create a custom grid; [block, below=of input] (model) Read start, goal, terrain, and treasure cells; [block, below=of model] (run) Run BFS and UCS independently; [block, below=of run] (paths) Reconstruct paths and combine treasure-route segments; [block, below=of paths] (metrics) Calculate cost, steps, expanded nodes, time, and score; [block, below=of metrics] (display) Visualize routes, frontier states, and comparison verdict; [arrow] (input) – (model); [arrow] (model) – (run); [arrow] (run) – (paths); [arrow] (paths) – (metrics); [arrow] (metrics) – (display);

Figure 1: High-level workflow of the pathfinding simulator.

7.2 Interactive Visualization

The interface supports preset selection and custom map creation. In custom mode, users can change the grid size, randomize the terrain, clear the map, and choose a paint tool for grass, mud, water, obstacles, start, goal, treasure, or erasing. Treasure values can be assigned using the available controls.

The simulation stores frames containing the current cell, visited cells, frontier contents, and partial path. Users can play, pause, step forward or backward, reset, and change playback speed. The display can show the BFS and UCS routes separately or together. The frontier inspector makes the internal data structures visible during execution.

8 Performance Measures

The simulator reports the following measures:

- **Total path cost:** Sum of terrain costs along the route.
- **Total steps:** Number of cell-to-cell moves.

- **Nodes expanded:** Number of states removed and processed during search, accumulated over route segments in treasure mode.
- **Execution time:** Measured using the runtime timer around the algorithm execution.
- **Treasure points:** Sum of the values of all collected treasures.
- **Net score:** Treasure points minus total movement cost.

The verdict component identifies the winner separately for path cost, steps, expanded nodes, and net score. This is important because one algorithm need not win every metric. In particular, BFS may use fewer steps while UCS incurs a lower movement cost.

9 Complexity Analysis

Let V be the number of traversable cells and E the number of valid adjacency edges in the grid.

For a single start-to-goal search:

- BFS runs in $O(V + E)$ time and uses $O(V)$ auxiliary space.
- UCS with a binary min-heap runs in $O((V+E)\log V)$ time and uses $O(V)$ space for the cost, parent, visited, and priority-queue structures.

For a four-neighbour grid, $E = O(V)$, so these bounds are commonly expressed as $O(V)$ for BFS and $O(V \log V)$ for heap-based UCS.

If there are t treasures and all orders are evaluated, there are $t!$ possible treasure sequences. The implementation caches pairwise segment searches to avoid repeatedly solving the same ordered pair, but permutation evaluation still grows factorially. For this reason, the implementation switches to a greedy ordering when the number of treasures exceeds six. The greedy method reduces route-order exploration but does not guarantee a globally optimal treasure order.

10 Expected Behaviour and Discussion

The central experiment is the comparison between a direct route through costly terrain and a longer route through cheaper terrain. In an environment where a swamp or water region lies on the geometrically shortest route, BFS may select that route because it minimizes the number of moves. UCS considers the cumulative terrain cost and may travel around the expensive region if the detour has a lower total cost.

When all traversable cells have equal cost, minimizing the number of steps and minimizing the total cost become equivalent, so BFS and UCS can return routes with the same objective value. When terrain costs differ, their routes may

differ. The outcome depends on the map layout and the placement of obstacles and treasures.

The project intentionally reports multiple metrics instead of describing one algorithm as universally faster or better. Execution time is influenced by the machine, runtime, map size, and implementation details. It should be measured over repeated runs for a rigorous timing comparison. Similarly, node expansions are useful for understanding search effort but do not alone determine wall-clock performance.

11 How to Run

11.1 React Interface

From the project directory, install dependencies and start the Vite development server: [language=bash,basicstyle=`,frame=single] npm install npm run dev` Open the local address printed by Vite in a web browser. The project also defines `npm run build` for creating a production build and `npm run preview` for previewing that build.

11.2 Python Streamlit Interface

Install the Python dependencies and launch the Streamlit application: [language=bash,basicstyle=`,frame=single] pip install -r requirements.txt streamlit run streamlit$_{a}$*pp.py* The application can then be opened using the local URL displayed in the terminal. For hosted deployment, the Streamlit application can be deployed through a Streamlit-compatible hosting service after adding the required dependencies and source files. The React interface can be deployed as a static Vite build on a compatible web host.

12 Limitations

The current implementation has several limitations:

1. Movement is restricted to four neighbouring cells; diagonal movement is not modelled.
2. Terrain costs are fixed constants rather than user-configurable values.
3. The exact treasure-order search is limited to six treasures; larger instances use a greedy approximation.
4. The simulator uses execution time from a single run for display. Repeated trials and statistical summaries would provide more reliable performance conclusions.
5. The application compares BFS and UCS only; algorithms such as Dijkstra's algorithm as a separately named baseline, A*, and bidirectional search are not included.

13 Conclusion

This project demonstrates how the choice of search strategy affects pathfinding in environments with non-uniform movement costs. BFS is suitable when the objective is to minimize the number of moves and all moves have equal cost. UCS is suitable when actions have different positive costs and the objective is to minimize total path cost. The treasure-hunt extension introduces a multi-goal planning problem and demonstrates how route ordering influences the final journey.

The interactive visualizations, frontier inspection, and performance audit make the algorithms easier to understand than a text-only implementation. The project also provides a useful foundation for future work, including A* search, configurable terrain costs, larger-scale treasure planning, repeated benchmark experiments, and deployment of the web interface.

Acknowledgment

The author acknowledges the use of open-source software libraries including React, TypeScript, Vite, Streamlit, NumPy, Matplotlib, and Python's standard-library data structures.

References

The updated references and three supporting figures are provided on the following pages.

SUPPORTING FIGURES

Conceptual illustrations prepared for this project report

Figure 1. Search problem formulation

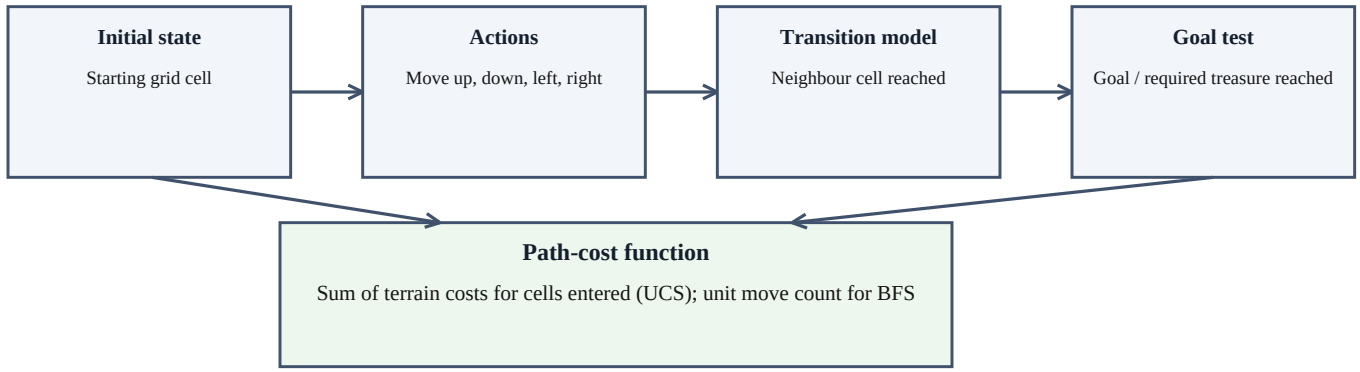
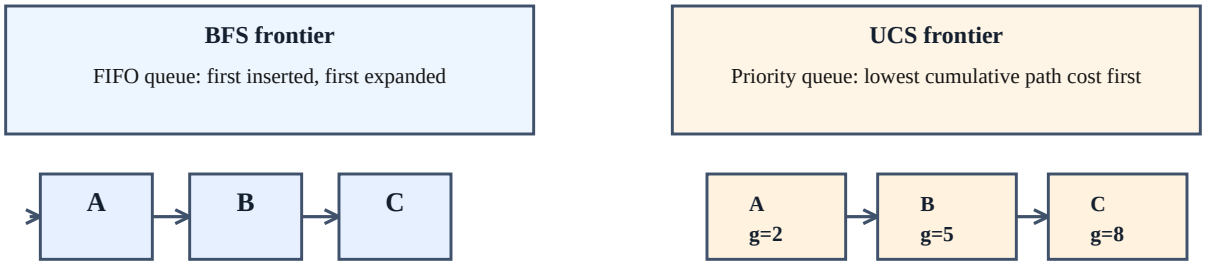


Figure 2. How BFS and UCS choose the next state

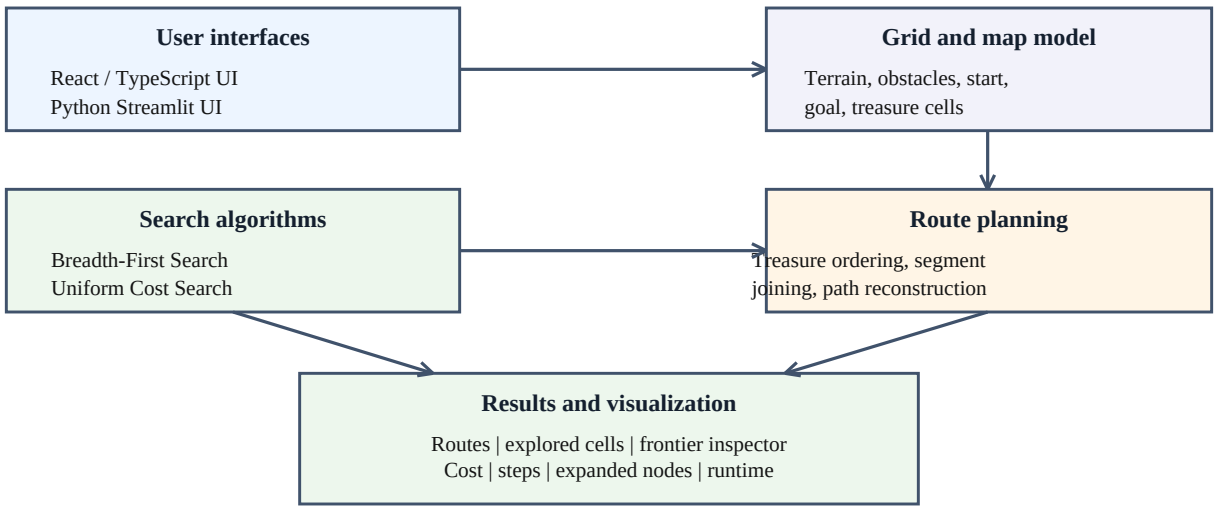


Expansion order is based on depth (number of moves). Terrain weights do not affect the queue order.

Expansion order is based on accumulated path cost. A longer route can be selected if it is cheaper.

Illustrative queue contents only; labels and costs are examples, not measured project results.

Figure 3. High-level project architecture



Architecture diagram is an original summary of the modules and workflow described in this project.

REFERENCES

Books, algorithm references, and visual learning resources

[1] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1, 269-271.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms (CLRS): chapters on breadth-first search and shortest paths.

[3] Breadth-first search. Wikipedia. https://en.wikipedia.org/wiki/Breadth-first_search

[4] Dijkstra's algorithm / Uniform-cost search. Wikipedia. https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm

[5] Red Blob Games. Introduction to the A* Algorithm: visual explanation of BFS, Dijkstra / UCS, and A*. https://www.redblobgames.com/pathfinding/a-star/introduction.html

[6] Russell, S., & Norvig, P. Artificial Intelligence: A Modern Approach (4th ed.), Chapter 3: Solving Problems by Searching.

Figures 1 and 2 are original conceptual diagrams informed by the search-problem formulation and search strategies described in Russell and Norvig, Chapter 3. Figure 3 is an original architecture summary based on this project implementation.